

A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is light green. They are positioned diagonally, with the blue one partially covering the green one.

Algoritma & Struktur Data

Week 6 - Sorting II



Last Week Review

- Sorting
 - Bubble sort
 - Represented by Bubble
 - Check every adjacent
 - Swap if it fulfill the condition
 - Selection sort
 - Get 1 value.
 - compare with the rest of element who have larger index
 - Swap
 - Continue
 - Insertion sort
 - Get 1 value.
 - Check predecessor
 - Slide it's index till found an index which fulfill swapping condition
 - Radix
 - Create a bucket
 - Linked list help
 - Get longest element
 - Put the bucket based on digit or char (number or string).



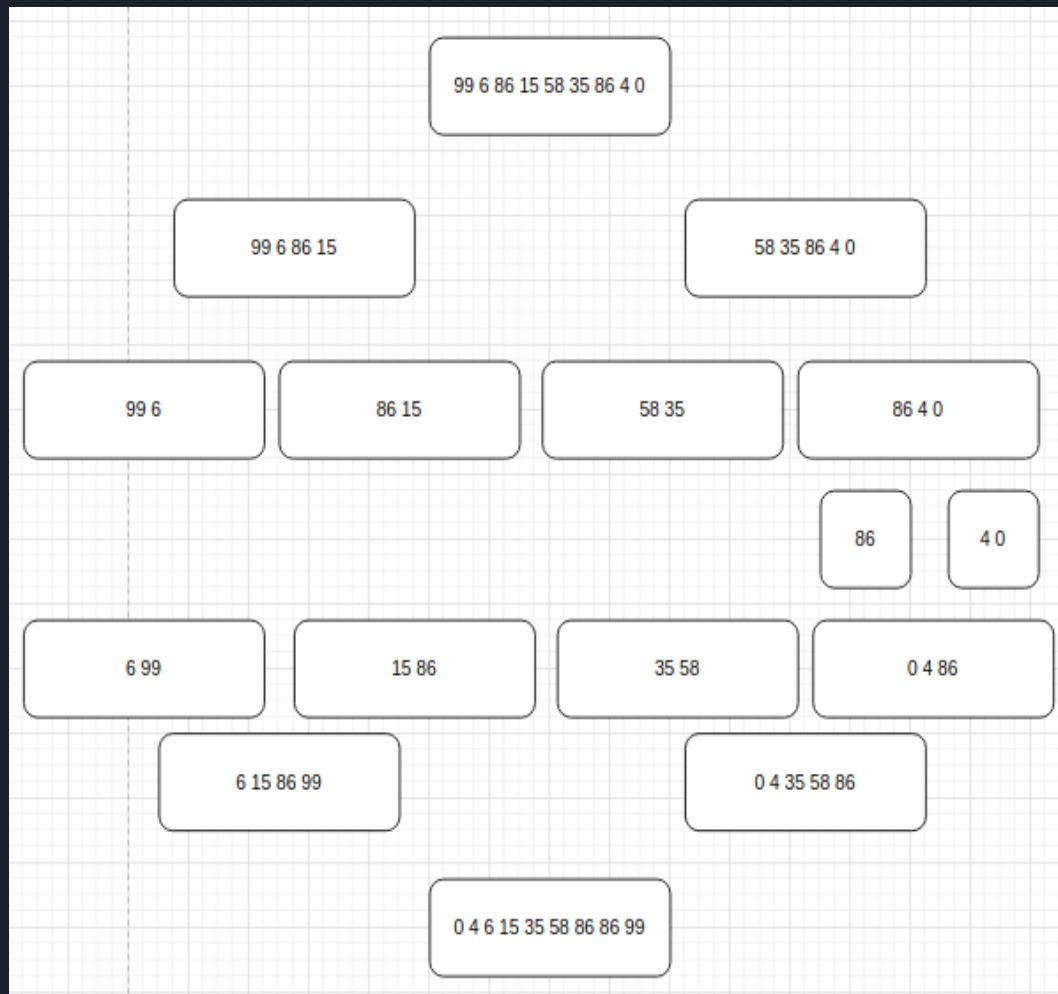
Outline

- Sorting
 - Merge Sort
 - Concept
 - Implementation
 - Quick Sort
 - Concept
 - Implementation
 - Shell Sort
 - Concept
 - Implementation
 - Heap Sort
 - Concept
 - Implementation



Merge Sort (Concept)

- Merge Sort Main Idea is divide and conquer
 - Split array to two section (smaller problem)
 - Each section will be keep splitted recursively
 - Ends the split if the member less or equal to 2
 - Merge the list from two section
 - Track element of both list.
 - Check which one is smaller/bigger.
 - Based on condition put it into newly created array.



Implementation

- Main mergesort function
 - Split One Array to two different Array
 - Sort half left
 - Sort half right
- Merge Function
 - Make Temporary Array
 - Put start index for both left and right data
 - Continuously compare data and assign it to the main array
 - Put remainder data to each respective section (left and right)

```
void mergeSort(int arr[], int l, int r)
{
    if (l < r) {
        int m = (l+r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
```

```
int main()
{
    int arr[] = { 9, 1, 2, 10, 100, 15, 20, 18, 17 };
    int arr_size = sizeof(arr) / sizeof(arr[0]);
    mergeSort(arr, 0, arr_size - 1);
    for(int i = 0; i < arr_size; i++){
        printf("%d ",arr[i]);
    }
    return 0;
}
```

```
void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int left_count = m - l + 1;
    int right_count = r - m;
    int L[r], R[r];
    for (i = 0; i < left_count; i++)
        L[i] = arr[l + i];
    for (j = 0; j < right_count; j++)
        R[j] = arr[m + 1 + j];
    i = 0;
    j = 0;
    k = l;

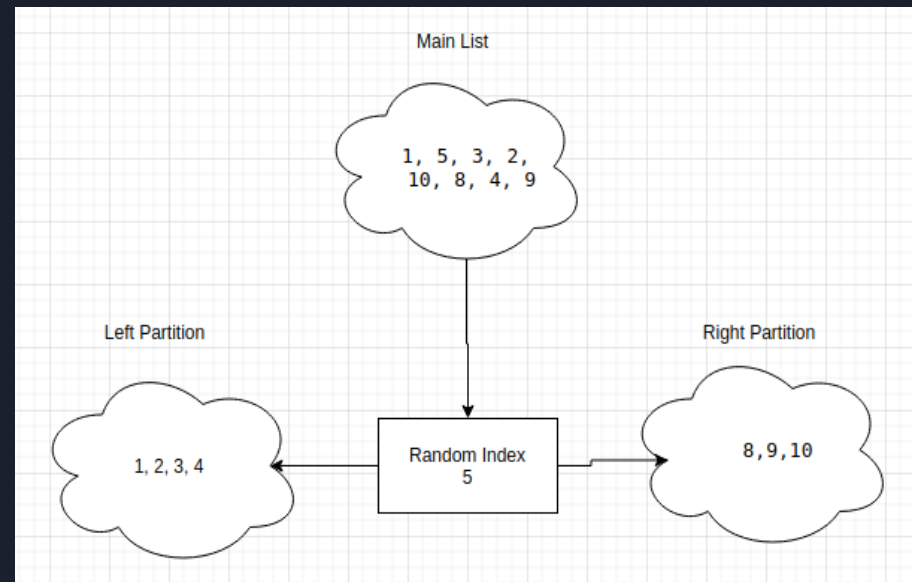
    for(; i < left_count && j < right_count; k++){
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
    }

    for(; i < left_count; i++,k++){
        arr[k] = L[i];
    }

    for(; j < right_count; j++,k++){
        arr[k] = R[j];
    }
}
```

Quick Sort (Concept)

- Quick Sort Main Idea is divide and conquer
 - Pick random index
 - Make Right and Left Partition
 - Right Partition is larger than value in random index
 - Left Partition is smaller than value in random index
 - Recursively Repeat the process on Left & Right



```
void q_sort(data, start, end){  
    if(start < end){  
        mid = make_part(data, start, end)  
        q_sort(data, start, mid-1)  
        q_sort(data, mid+1, end)  
    }  
}
```

Implementation

- Main quicksort function
 - Make two partition
 - Get the point between partition
 - Recursively quick sort both partition.
- Partition Function
 - Basically swap on the left and right.
 - Similar with selection
 - Return the final piv

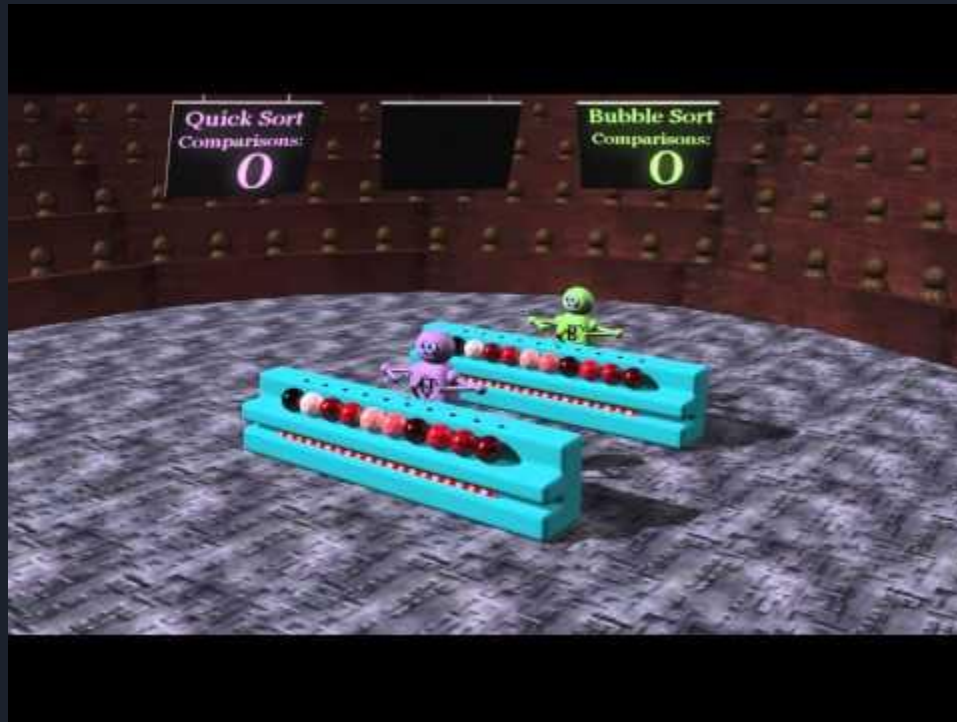
```
#include <stdio.h>

int partition(int data[], int start, int end) {
    int pivot = data[end]; // get last element
    int i = start; // left side index
    int temp;
    for(int j = start; j < end; j++){
        if(data[j] > pivot){
            temp = data[j];
            data[j] = data[i];
            data[i] = temp;
            i++;
        }
    }
    temp = data[i];
    data[i] = data[end];
    data[end] = temp;
    return i;
}

void q_sort(int data[], int start, int end){
    if(start < end){
        int piv = partition(data, start, end);
        q_sort(data, start, piv-1);
        q_sort(data, piv+1, end);
    }
}

int main(){
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    q_sort(arr, 0, n - 1);
    for(int i = 0; i < n; i++){
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Visualization



Shell Sort(Concept)

- Shell sort is an improvement of insertion sort.
 - Modify the gap used to check the value
 - Instead of adjacent we enable checking between different gap.
 - Every pass will make the gap smaller.

第一遍



Implementation

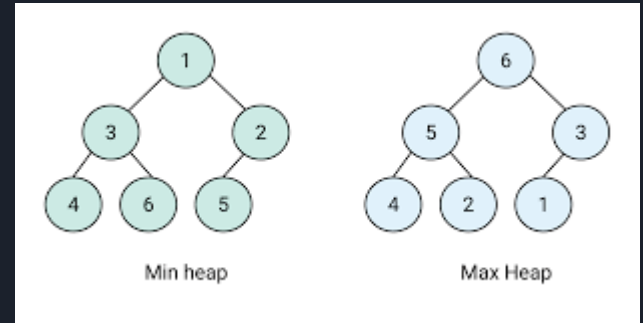
- The first “For” Set the gap in the middle ($N/2$) or maybe $N/(1.5)$ larger gap is possible.
- The second “For” works as an insertion sort, instead starting from 1, now it start from gap.
- The third “For” works as a checking mechanism. Instead of adjacent now we check element with certain position gap.

```
int shellSort(int arr[], int n){
    for (int gap = n/2; gap > 0; gap /= 2){
        for (int i = gap; i < n; i += 1){
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
                arr[j] = arr[j - gap];
            arr[j] = temp;
        }
    }
    return 0;
}
```

```
int main()
{
    int arr[] = { 9, 1, 2, 10, 100, 15, 20, 18, 17 };
    int arr_size = sizeof(arr) / sizeof(arr[0]);
    shellSort(arr, arr_size );
    for(int i = 0; i < arr_size; i++){
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Heap Sort(Concept)

- Heap Sort is a sorting algorithm which possible by using a heap tree data structure
- Heap tree is a complete Binary Tree.
- Heapify is the process of heap tree construction.
- There is two type of Heap Tree
 - Min Heap : Heap tree where the parent node is smaller or equal to either of it's child
 - Max Heap : Heap tree where the parent node is smaller or equal to either of it's child
- Special Implementation:
 - If you have continuous stream of number, and for every insertion you wanted to know the n lowest number.
 - Useful for ordering live and ongoing data.



Implementation (HEAPIFY)

- Imagine the array was represented as a tree this way
- Heapify Algorithm
 1. Get First element (index i)
 2. Get Left Child Element ($i*2 + 1$)
 3. Get Right Child Element ($1*2 + 2$)
 4. Check if the child element is larger/smaller than parent.
 5. If so. Swap and recursively do heapify.
- To create a complete Heap tree
 - Basically you need to get the most bottom left parent. $((n/2)-1)$
 - Heap it all to the top. It will ensure all sub tree will have proper heap structure

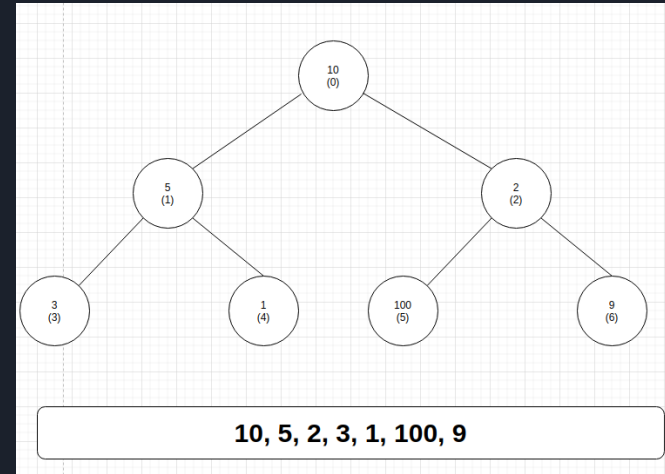
```
void max_heap(int *data, int idx, int n){
    int largest = idx;
    int left = 2*idx+1;
    int right = 2*idx+2;

    if (left < n &&
        data[left] > data[largest])
        largest = left;
    if (right < n &&
        data[right] > data[largest])
        largest = right;

    if (largest != idx){
        int temp = data[largest];
        data[largest] = data[idx];
        data[idx] = temp;
        max_heap(data, largest, n);
    }
}
```

```
int * build_heap(int *data, int n)
{
    int i;
    for (i=(n/2)-1; i >= 0; i--) {
        max_heap(data, i, n);
    }
    return data;
}
```

Contoh Unsorted Tree



Heap Sort Implementation

- Heap sort follow these instructions:
 - Set element N as the number of valid element
 - Get the first root element (Index 0)
 - Swap the first root element and element at index N
 - we can ignore those last N element
 - Since the last N element will be filled with N biggest/smallest element,
 - So reduce N by 1 (ignoring newly swapped elements).
 - Heap the root again to reorder the heap tree.
 - Repeat.

```
void heap_sort(int* data, int size)
{
    data = build_heap(data, size);

    while (size > 1)
    {
        int temp = data[0];
        data[0] = data[size-1];
        data[size-1] = temp;
        size--;
        max_heap(data, 0, size);
    }
}
```



THE END